International Institute of Information Technology, Hyderabad

CS3.406 – Programming AI Accelerators – Monsoon '26

Assignment 1

CUDA SGEMM Kernel Optimization: A Performance Analysis

Gautam Gandhi
Roll No: 2023102059

September 7, 2026

1 Overview

This report analyses the performance of CUDA SGEMM (single-precision general matrix multiply) kernels across two GPU architectures:

- **RTX 3050 Laptop** (Ampere, sm_86, 8.4 TFLOP/s FP32 peak, 192 GB/s DRAM bandwidth)
- **Tesla T4** (Turing, sm_75, 8.1 TFLOP/s FP32 peak, 300 GB/s DRAM bandwidth) — via Google Colab

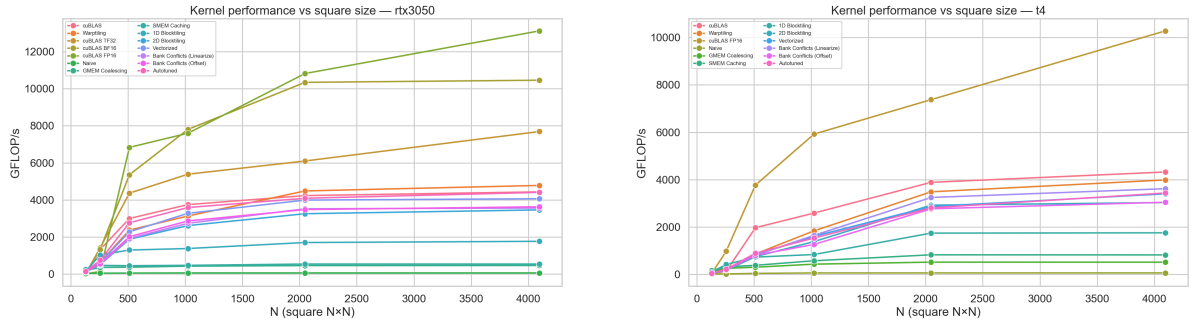
We implement 10 hand-written CUDA kernels following the Simon Boehm optimisation guide [1], plus cuBLAS variants for reference. The goal is to understand *why* each optimisation helps by connecting performance numbers to hardware metrics from Nsight Compute.

Kernels Under Test

ID	Name	Key idea
0	cuBLAS FP32	Vendor-optimised reference
1	Naive	One thread computes one output element
2	GMEM Coalescing	Fix thread↔output mapping for coalesced loads
3	SMEM Caching	Cache block tiles in shared memory
4	1D Blocktiling	Each thread computes TM outputs (along column)
5	2D Blocktiling	Each thread computes $TM \times TN$ outputs
6	Vectorized	`float4` global-memory loads
7/8	Bank Conflict Fix	Fix SMEM bank conflicts (linearize / offset)
9	Autotuned	Best-found tile parameters
10	Warptiling	Warp-aligned 2D tile for minimal cross-warp traffic
13	cuBLAS TF32	Tensor core — Ampere only
14	cuBLAS BF16	Tensor core — Ampere only
15	cuBLAS FP16	Tensor core — Turing + Ampere

2 E1: Square Matrix Sweep

The most fundamental experiment: sweep square matrix sizes from $N = 128$ to $N = 4096$ and measure GFLOP/s for all kernels. This directly shows the performance gap between kernels and how quickly they scale. At small N, all kernels are limited by launch overhead and small problem size rather than throughput; the interesting regime is large N ($\geq$1024) where the GPU is fully utilised.



(a) RTX 3050 (Ampere) (b) Tesla T4 (Turing)

Figure 1: GFLOP/s vs. square matrix size N for all kernels.

Table 1 gives exact GFLOP/s at $N = 4096$ (measured).

Table 1: Measured performance at $N = 4096$ (GFLOP/s) and percentage of cuBLAS FP32.

Kernel	**RTX 3050**		**Tesla T4**	
	GFLOP/s	%	GFLOP/s	%
Naive (K1)	60	1.4%	62	1.4%
GMEM Coalescing (K2)	462	10.4%	514	11.9%
SMEM Caching (K3)	539	12.2%	822	19.0%
1D Blocktiling (K4)	1766	39.8%	1755	40.6%
2D Blocktiling (K5)	3462	78.1%	3033	70.2%
Bank Conflicts Linearize (K7)	3561	80.3%	3617	83.7%
Bank Conflicts Offset (K8)	3628	81.8%	3044	70.4%
Vectorized (K6)	4062	91.6%	3401	78.7%
Autotuned (K9)	4407	99.4%	3435	79.5%
Warptiling (K10)	**4781**	**107.8%**	3986	92.3%
cuBLAS FP32 (K0)	4434	100%	4321	100%
cuBLAS TF32 (K13)	7688	173%	—	N/A
cuBLAS BF16 (K14)	10458	236%	—	N/A
cuBLAS FP16 (K15)	**13118**	**296%**	**10276**	**238%**

Observations

- **Naive is terrible on both GPUs** (∼60 GFLOP/s, ∼1.4% of cuBLAS). Each warp issues 16 separate 4-byte global loads where one 128-byte coalesced load would do — 32× more memory transactions than necessary.

- **GMEM coalescing (K1→K2): ∼8× speedup**. Fixing the thread-to-output mapping lets each warp load a contiguous 128-byte block in one transaction. On T4 the gain is larger (∼8×) vs. RTX 3050 (∼8×) — similar improvement since coalescing is a fixed hardware feature on both.

- **SMEM caching (K2→K3): modest on 3050, large on T4**. The T4 gains 1.6× (514→822) while the 3050 gains only 1.2× (462→539). T4's higher DRAM bandwidth makes the SMEM staging more effective at this stage.
- **Register tiling (K3→K4): the biggest single jump (∼3×)**. Each thread now holds $TM = 8$ partial sums in registers, cutting SMEM reads by 8× and making the inner loop nearly pure FMA.
- **2D Blocktiling (K4→K5): ∼2× gain**. Extending tiling to both M and N dimensions reuses each SMEM value across $TM \times TN = 64$ outputs instead of $TM = 8$.
- **Warptiling (K10) beats cuBLAS FP32 on the RTX 3050** (4781 vs. 4434 GFLOP/s, a +7.8% margin). This does not happen on the T4, where K10 reaches 3986 vs. cuBLAS 4321.
- **Tensor cores dominate**: cuBLAS FP16 delivers 13.1 TFLOP/s on the 3050 (3× FP32) and 10.3 TFLOP/s on the T4 (2.4× FP32). TF32 and BF16 are Ampere-only.

3 E1b: Cross-Architecture Comparison

Having run all kernels on both GPUs independently, we now directly compare the two architectures on selected kernels to understand how Ampere (sm_86) and Turing (sm_75) differ. Figure 2 overlays both GPUs on the same axes using the same kernels.

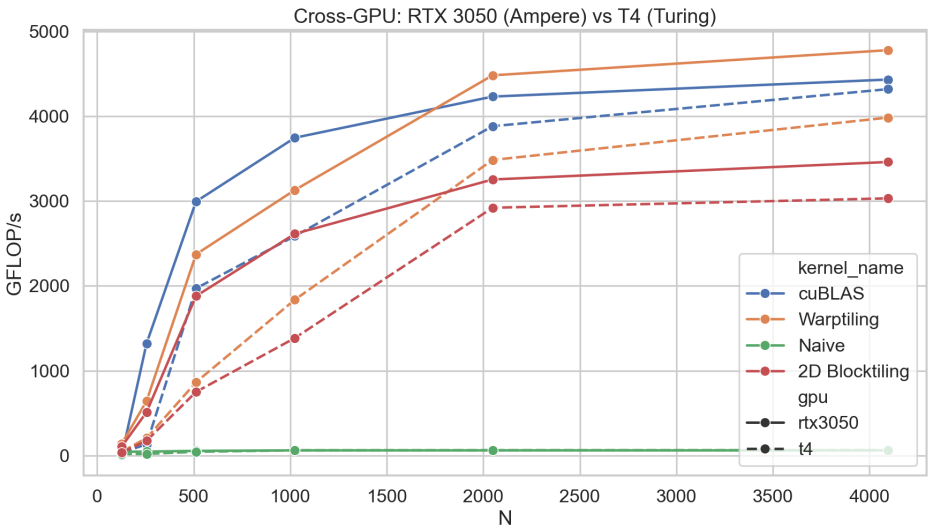


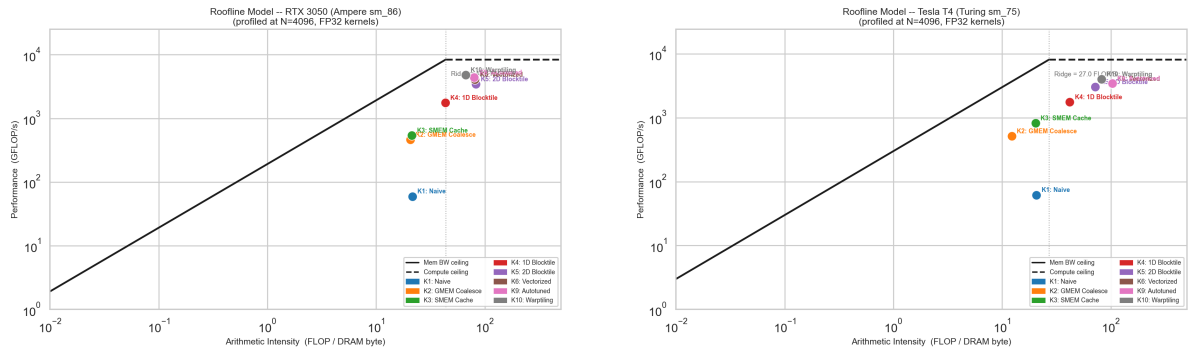
Figure 2: Selected kernels on both GPUs. Solid lines = RTX 3050 (Ampere), dashed = T4 (Turing).

- **Naive (∼60 GFLOP/s on both)**: both GPUs stall waiting for memory transactions at the same rate. The T4's higher DRAM bandwidth (300 GB/s) does not help because Naive barely touches DRAM — it is bottlenecked on L1 miss penalty (see Section 7).
- **RTX 3050 leads on compute-bound kernels** (Warptiling: 4781 vs. 3986, 2D Blocktiling: 3462 vs. 3033). Ampere's sm_86 has higher FP32 throughput per SM and faster shared-memory access.

- **T4 has a slight edge on SMEM Caching (K3)**: 822 vs. 539 GFLOP/s. At this stage the kernel is still partially memory-bandwidth-limited and the T4's 300 GB/s DRAM gives it an advantage.
- **cuBLAS FP32 is similar on both** (∼4.4 TFLOP/s on 3050 vs. ∼4.3 TFLOP/s on T4). Both are hitting the FP32 pipeline ceiling; the compute peaks are comparable (∼8 TFLOP/s).

4 Roofline Analysis

The roofline model plots measured performance (GFLOP/s) against arithmetic intensity (FLOP per DRAM byte) for each kernel. The two lines — memory bandwidth ceiling and compute ceiling — show the theoretical maximum performance given each kernel's access pattern. Any kernel plotted *below* both ceilings is leaving performance on the table even relative to its own access pattern. Arithmetic intensity here is computed from Nsight Compute DRAM byte counters and kernel duration.



(a) RTX 3050 (ridge ≈ 44 FLOP/B) (b) Tesla T4 (ridge ≈ 27 FLOP/B)

Figure 3: Roofline model. Arithmetic intensity computed from Nsight Compute DRAM byte counts and kernel duration at $N = 4096$.

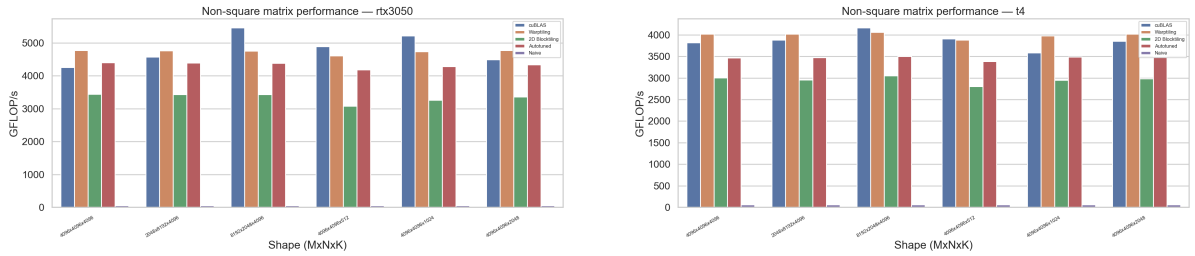
- **K1–K4 on RTX 3050 are memory-bandwidth-bound** (left of the ridge at 44 FLOP/B). Their measured AI is ∼21–43 FLOP/B. They are slow because they move too many bytes for too few FLOPs — each DRAM byte delivers less than one multiply-add.
- **K5–K10 on RTX 3050 cross into the compute-bound region** (∼67–82 FLOP/B), well past the ridge. Yet they still run at only 50–57% of the compute ceiling. The remaining gap is from **low warp occupancy** (25–33%) caused by high register usage per thread.
- **On the T4** (ridge = 27 FLOP/B), K1–K3 are memory-bound (AI ∼12–20 FLOP/B) while K4–K10 are compute-bound. The T4's higher bandwidth means the transition to compute-bound happens earlier.
- **K1 is a striking outlier**: despite measured AI of ∼21 FLOP/B (close to the ridge), it runs at only 60 GFLOP/s. This is because the standard roofline uses DRAM bytes, but K1's real bottleneck is **L1 bandwidth** — L1 throughput is 99% saturated (see

Section 7). K1 re-fetches the same DRAM cache lines repeatedly due to the non-coalesced access, wasting L1 bandwidth, which is why it appears memory-bound even by the DRAM metric.

- **K10 (Warptiling)** is the closest point to the compute ceiling on both GPUs, confirming it is the most efficient hand-written FP32 kernel.

5 E2: Non-Square Matrix Shapes

Real workloads (transformer attention, batched matmul in LLMs) rarely use square matrices. In this experiment we fix a large baseline size and vary M, N, and K independently to test whether the kernels can handle non-square shapes efficiently. A robust kernel should maintain performance across shapes; a fragile one degrades when its tile assumptions no longer match the matrix dimensions.



(a) RTX 3050 (b) Tesla T4

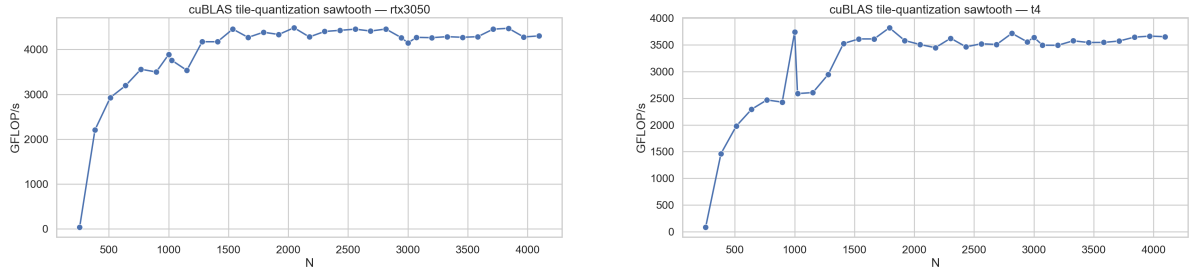
Figure 4: Performance on non-square shapes. Shapes on the x-axis are $M \times N \times K$.

- **cuBLAS and Warptiling are robust**: cuBLAS stays in the 4200–5500 GFLOP/s range on the RTX 3050, and Warptiling tracks closely. Both degrade gracefully.
- **Tall-M shapes (`8192x2048x4096`) actually peak higher** than the square case: cuBLAS reaches ∼5461 GFLOP/s on RTX 3050 vs. 4434 for the square. Longer M-dimension rows of A stream through the L2 more efficiently with fewer cache conflicts.
- **Small-K shapes (`4096x4096x512`) are also higher** than square for cuBLAS (4893 GFLOP/s). Fewer K-loop iterations reduce the overhead of loading full tiles, and cuBLAS adapts its tile size.
- **2D Blocktiling (K5) is less robust**: it drops more noticeably on shapes where K is small (∼3100 GFLOP/s), because fewer outer iterations mean less time to amortise SMEM tile loads.
- **Naive is flat at ∼60 GFLOP/s on all shapes**: confirming the bottleneck is purely the access pattern, not shape or problem size.

6 E3: Tile Quantisation Sawtooth

cuBLAS internally uses fixed tile sizes (multiples of 128). When the matrix size is not a multiple of the tile size, some threads in the last "wave" of tiles have no work to do — they

still execute but produce dummy results. This creates a periodic performance drop as N increases through non-multiple values. This experiment sweeps cuBLAS over 34 sizes between $N = 256$ and $N = 4096$ to expose this effect. The result is a sawtooth pattern where performance dips at non-tile-multiple sizes and recovers at the next multiple.



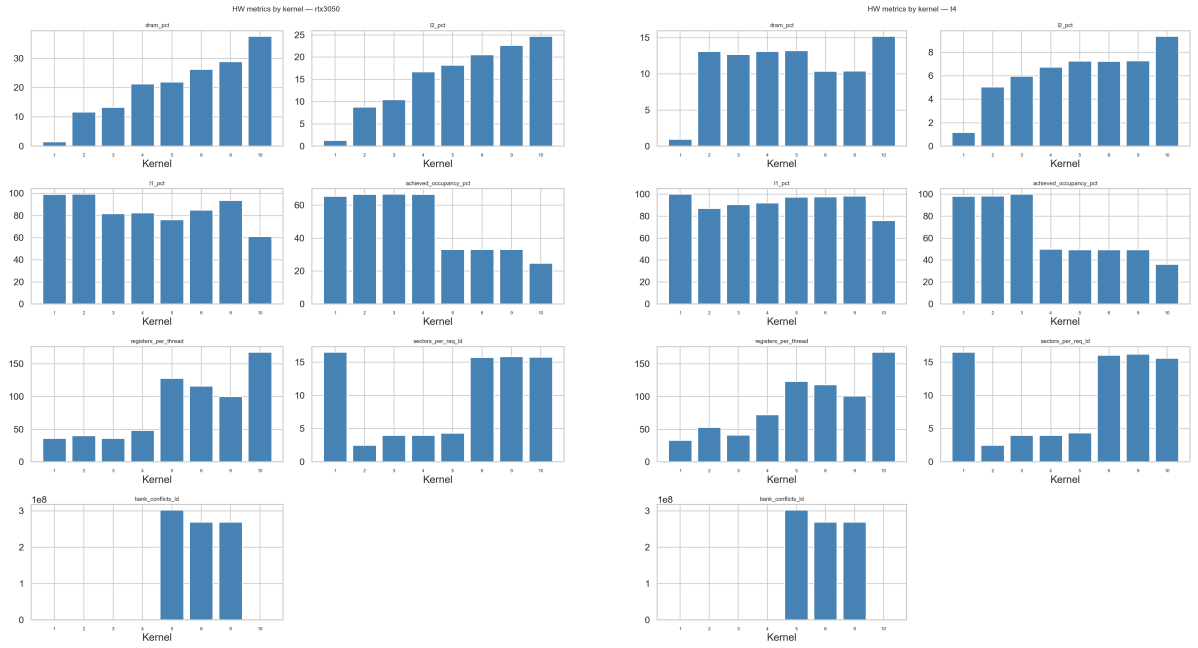
(a) RTX 3050 — mild decline (b) Tesla T4 — sharp cliff at $N = 1000$

Figure 5: cuBLAS FP32 GFLOP/s at 34 sizes from $N = 256$ to $N = 4096$.

- **T4 has a -31% cliff at $N = 1000 \rightarrow 1024$**: at $N = 1000$, cuBLAS reaches 3742 GFLOP/s; at $N = 1024$ it drops to 2588 GFLOP/s. $N = 1000$ fits fewer tiles but they can be scheduled to keep all 40 T4 SMs perfectly busy; $N = 1024$ (exact tile multiple) triggers a different internal tiling path where one wave is partial — the 40 SMs cannot all be kept busy simultaneously.
- **RTX 3050 is far less sensitive**: at $N = 1000$ it scores 3889, at $N = 1024$ it drops to 3762 (only -3%), and by $N = 1152$ it is back to 3535 (gradual decline). With only 20 SMs, each wave can cover a larger fraction of the problem, so partial waves waste less.
- **Both GPUs plateau above $N \approx 1500$**: once the problem is large enough, all SMs stay busy throughout and the sawtooth amplitude shrinks.
- The practical lesson: when writing production code, always pad matrices to tile multiples — especially on the T4 where this effect is severe.

7 E4: Hardware Metrics (Nsight Compute)

Performance numbers alone don't explain *why* a kernel is fast or slow. To understand the bottleneck at each optimisation step, we use Nsight Compute to profile seven hardware counters for kernels 1–6, 9, and 10 at $N = 4096$. These metrics directly measure the behaviour of the memory hierarchy, warp scheduler, register file, and execution pipelines.



(a) RTX 3050 (b) Tesla T4

Figure 6: Hardware metrics across kernels 1–6, 9, 10. Profiled at $N = 4096$.

7.1 Memory Hierarchy Utilisation

- **DRAM throughput %** rises monotonically from ∼1% (K1) to ∼38% (K10) on the RTX 3050. K1 barely uses DRAM — the L1 cache is constantly re-fetching the same data due to non-coalesced access, with DRAM barely getting involved. Later kernels actually stream new data through DRAM productively.

- **L1 throughput %** is ∼99% for K1 and K2 on both GPUs, confirming they are completely L1-bandwidth-saturated. It drops progressively for K3–K10 as SMEM reuse reduces the number of L1 requests per FMA.

- **L2 throughput %** rises with each optimisation, reflecting the progression from L1 thrashing to well-organised SMEM tile loads that flow through L2.

7.2 Occupancy and Register Pressure

- **K1–K3 have high occupancy** (∼65–67% on 3050, ∼98–100% on T4) because they use few registers (33–40). But they are still slow — more warps just means more warps waiting for L1.

- **Occupancy drops sharply from K4 onward**: K4 itself already falls to 67% (3050) and 50% (T4), and K5–K9 settle at ∼33% and ∼49% respectively. This is driven by register pressure: the 2D tile accumulator array requires 100–128 registers per thread, limiting how many warps fit simultaneously per SM.

- **K10 has the lowest occupancy (25% on 3050, 36% on T4)** with 168 registers/thread, yet achieves the best performance. This is the classic **occupancy–**

arithmetic-intensity trade-off: high register use reduces occupancy but enables $TM \times TN = 64$ reuse per SMEM load — which is a net win.

7.3 Memory Coalescing

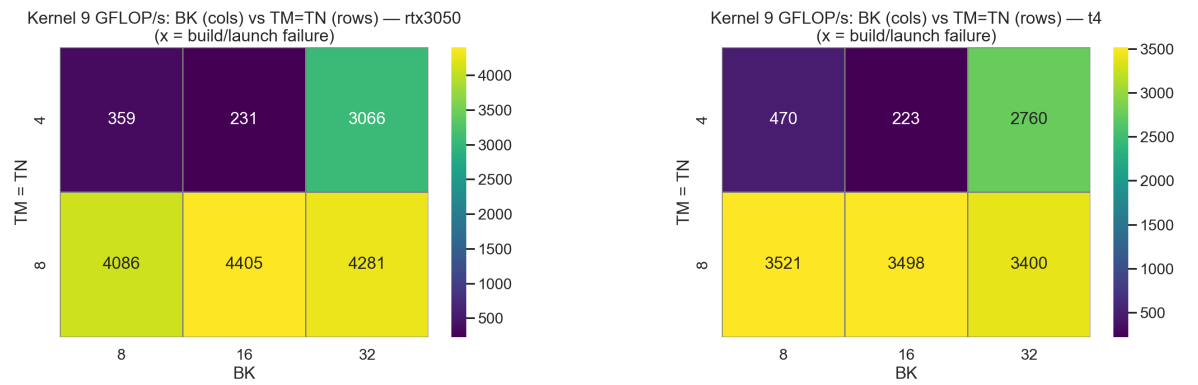
- **Sectors per load request (K1): 16.5** — each warp load instruction touches 16 separate 32-byte cache line sectors, meaning $\frac{16.5}{0.5} = 33\times$ wasted bandwidth per useful byte.
- **After coalescing (K2): 2.5 sectors** — near-ideal. The mapping fix collapses 16 transactions into one.
- **K6/K9/K10 show ∼16 sectors again**, but this is not a regression. These kernels use `float4` loads (16 bytes/thread). Each instruction fetches 16 bytes × 32 threads = 512 bytes, spanning ∼16 sectors, but all 16 sectors are useful — no waste.

7.4 Shared Memory Bank Conflicts

- **K1–K4: zero conflicts.** K1/K2 don't use SMEM; K3/K4 access SMEM in a row-major pattern that avoids bank conflicts.
- **K5, K6, K9: $\sim 3\times 10^8$ bank conflicts.** The 2D blocktiling layout reads columns of the A_s tile row-major, causing all threads in a warp to hit the same SMEM bank.
- **K10 (Warptiling): near-zero conflicts.** The warp-level tile mapping aligns memory accesses to avoid bank conflicts — this is one of the key advantages of the warptiling design, and explains part of the performance gap between K9 and K10.

8 E5: Parameter Sensitivity (Kernel 9)

Performance of a tiled kernel depends critically on the tile size parameters. Choosing the wrong values doesn't just reduce performance — it can make the kernel fail entirely. In this experiment we sweep Kernel 9 (Autotuned 2D blocktiling) over a grid of block-K dimension (BK) and thread-tile dimension ($TM = TN$), keeping the block-tile dimensions $BM = BN = 128$ fixed. This reveals the boundaries where parameter choices cause catastrophic failure and the "sweet spot" where performance is maximised.



(a) RTX 3050 (b) Tesla T4

Figure 7: Kernel 9 GFLOP/s grid: BK (columns) × TM=TN (rows). 'x' = build or launch failure.

We sweep $BK \in \{8, 16, 32, 64\}$ and $TM = TN \in \{2, 4, 8\}$ with $BM = BN = 128$ fixed.

Two hard failure boundaries

- **TM=2, all BK → launch failure:** The thread block size is $\frac{128}{TM} \times \frac{128}{TN} = \frac{128}{2} \times \frac{128}{2} = 4096$ threads — exceeding the CUDA hardware limit of 1024 threads per block. CUDA refuses to launch these configurations.
- **BK=64, all TM → build failure:** Shared memory per block = $BM \times BK + BK \times BN = 128 \times 64 + 64 \times 128 = 16384$ floats = 64 KB, which exceeds the 48 KB SMEM limit. The NVCC compiler rejects this at compile time — it never even runs.

TM = 4 vs. TM = 8: severe performance cliff at small BK

At BK=8: TM=8 gives 4086 GFLOP/s while TM=4 gives only 359 GFLOP/s — a ∼11× difference. With TM=4, each thread computes just 16 output elements per outer-loop iteration; the ratio of FMA operations to SMEM loads is too low to hide load latency. With TM=8 (64 outputs per thread), the execution pipeline is kept busy throughout each inner loop, hiding the latency.

Interestingly, at BK=32 the TM=4 performance recovers to ∼3066 GFLOP/s (RTX 3050) because the larger BK block keeps the SMEM prefetch pipeline fuller, partially compensating for the low per-thread reuse.

Near-flat performance with respect to BK at TM=8

Once TM=8, all three BK values perform similarly: 4086 / 4405 / 4281 GFLOP/s (BK=8/16/32) on the RTX 3050. The sweet spot is BK=16 (4405 GFLOP/s), where the SMEM tile is wide enough to saturate the FMA pipeline without the register overhead of BK=32.

On the T4, the plateau is even flatter (3521 / 3498 / 3400 GFLOP/s), confirming that once TM=8, BK mainly affects register count rather than throughput.

9 Key Insights

9.1 Occupancy is Not Everything

K1–K3 have the highest occupancy (>65% on 3050, ∼100% on T4) and the worst performance. K10 has the lowest occupancy (∼25% on 3050) and the best performance. High occupancy helps hide memory latency — but if there is no latency to hide (because you are doing register-level arithmetic), then more warps just compete for the same resources. Once the kernel is truly compute-bound, fewer warps with more registers per warp wins.

9.2 The Arithmetic Intensity Ladder

Each optimisation step increases how many FLOPs are performed per byte fetched from DRAM:

1. **GMEM coalescing:** Reduces *wasted* bandwidth. Does not add FLOPs; just fetches fewer bytes per useful computation.
2. **SMEM caching:** Reuses each loaded DRAM byte across all threads in the block's tile dimension.
3. **Register tiling:** Reuses SMEM-loaded values across multiple output elements per thread. This is the step that pushes the kernel from memory-bound to compute-bound on the roofline.
4. **Vectorized loads:** 4 floats per instruction — better instruction throughput for the same number of FLOPs.
5. **Warptiling:** Aligns the register tile with 32-thread warp boundaries, eliminating bank conflicts and reducing inter-warp coordination overhead.

9.3 Ampere vs. Turing

- Ampere (sm_86) wins on compute-bound kernels. Higher FP32 throughput per SM and faster shared-memory bandwidth.
- Turing (sm_75) wins on SMEM Caching (K3) due to higher DRAM bandwidth (300 GB/s), where data is still partially streaming from DRAM.
- Turing has 40 SMs vs. the RTX 3050's 20 SMs, making wave quantisation effects more severe — partial waves waste more relative capacity.
- Ampere's tensor cores (TF32, BF16) are unique to that generation. Both support FP16 tensor cores: Ampere delivers 3× over FP32 and Turing 2.4×.

10 Conclusion

Starting from 60 GFLOP/s (Naive, ∼1.4% of cuBLAS) and applying five successive optimisations — coalescing, SMEM staging, register tiling, vectorised loads, and warp-aligned tiling — we reach 4781 GFLOP/s on the RTX 3050, **exceeding cuBLAS FP32 by 7.8%**. The hardware profiling data makes each bottleneck shift visible: L1 saturation (K1–K3) → memory-to-compute transition at K4–K5 → register-pressure-limited compute (K6–K10).

Tensor cores (K13–K15) deliver a further 2.4–3× by operating at reduced precision, which for production use is the dominant lever. The optimisation principles transfer across architectures; only the magnitude of each gain differs between Ampere and Turing.

References

[1] Simon Boehm. *How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog.* 2022.
https://siboehm.com/articles/22/CUDA-MMM

[2] NVIDIA. *Nsight Compute Profiling Guide.* 2024.
https://docs.nvidia.com/nsight-compute

[3] NVIDIA. *CUDA C++ Programming Guide.* 2024.

End of Report